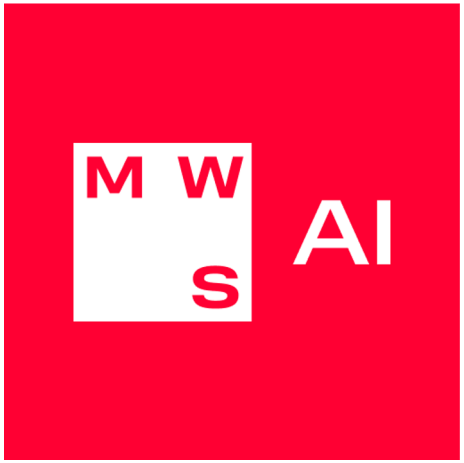
Alternative LLMs

Valentin Malykh



A Few Words about Me

- 10+ years of experience in AI and especially in NLP
- PhD in Natural Language Processing
- ex-Yandex, ex-VK, ex-Huawei, now at MWS AI
- 7+ years of giving courses in NLP

Agenda

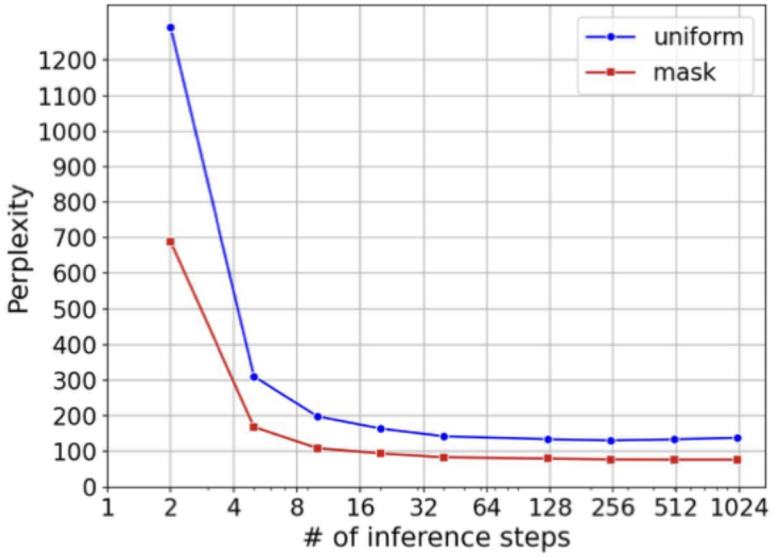
- Diffusion LMs
- State Space Models
- New RNNs
- Exotic LMs

Diffision LMs

Diffusion for Text

```
Janet's ducks lay 16 eggs per day. She eats three for breakfast
every morning and bakes muffins for her friends every day with
four. She sells the remainder at the farmers' market daily for $2
per fresh duck egg. How much in dollars does she make every day at
the farmers' market?
```

Diffusion for Text



t = 128 **[MASK] [MASK] [MASK] [MASK] [MASK] [MASK]...**
t = 25 In response **[MASK]** the demands , **[MASK] [MASK]**y Workers union said **[MASK]** backflow fund **[MASK]**s would face further investigation and a fine.
t = 0 In response to the demands , the Community Workers union said the backflow fund managers would face further investigation and a fine .

Original: Caterpillar is eager to expand in Asia , where it trails local competitors such as Komatsu Ltd
Corrupted: Caterpillar is eager to expand in **[MASK]** , **[MASK]** it **[MASK]** s local competitors such as Komatsu Ltd
Reconstructed: Caterpillar is eager to expand in China , where it faces local competitors such as Komatsu Ltd

Diffusion

Forward Process

$$q(\boldsymbol{x}_{1:T}|\boldsymbol{x}_0) = \prod_{t=1}^{T} q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$$

Reverse Process

$$p_\theta(\boldsymbol{x}_{0:T}) = p(\boldsymbol{x}_T) \prod_{t=1}^{T} p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)$$

Diffusion for Text

Variable Lower Bound Loss

$$L_{\text{vb}} = \mathbb{E}_{q(\boldsymbol{x}_0)} \Bigg[\underbrace{D_{\text{KL}}[q(\boldsymbol{x}_T|\boldsymbol{x}_0)||p(\boldsymbol{x}_T)]}_{L_T} + \sum_{t=2}^{T} \underbrace{\mathbb{E}_{q(\boldsymbol{x}_t|\boldsymbol{x}_0)} \left[D_{\text{KL}}[q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)||p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)] \right]}_{L_{t-1}} \underbrace{-\mathbb{E}_{q(\boldsymbol{x}_1|\boldsymbol{x}_0)}[\log p_\theta(\boldsymbol{x}_0|\boldsymbol{x}_1)]}_{L_0} \Bigg].$$

Diffusion for Text

Forward Process

$$q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) = \mathcal{N}\left(\boldsymbol{x}_t|\sqrt{1-\beta_t}\boldsymbol{x}_{t-1}, \beta_t\boldsymbol{I}\right)$$

Reverse Process

$$p_\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = \mathcal{N}\left(\boldsymbol{x}_{t-1}|\boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t), \boldsymbol{\Sigma}_\theta(\boldsymbol{x}_t, t)\right)$$

Diffusion for Text

$$q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) = \mathrm{Cat}(\boldsymbol{x}_t; \boldsymbol{p} = \boldsymbol{x}_{t-1}\boldsymbol{Q}_t)$$

where $$[\boldsymbol{Q}_t]_{ij} = q(x_t = j|x_{t-1} = i)$$

Diffusion for Text

$$q(\boldsymbol{x}_t|\boldsymbol{x}_0) = \text{Cat}\left(\boldsymbol{x}_t; \boldsymbol{p} = \boldsymbol{x}_0\overline{\boldsymbol{Q}}_t\right), \quad \text{with} \quad \overline{\boldsymbol{Q}}_t = \boldsymbol{Q}_1\boldsymbol{Q}_2\ldots\boldsymbol{Q}_t$$

$$q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) = \frac{q(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}, \boldsymbol{x}_0)q(\boldsymbol{x}_{t-1}|\boldsymbol{x}_0)}{q(\boldsymbol{x}_t|\boldsymbol{x}_0)} = \text{Cat}\left(\boldsymbol{x}_{t-1}; \boldsymbol{p} = \frac{\boldsymbol{x}_t\boldsymbol{Q}_t^\top \odot \boldsymbol{x}_0\overline{\boldsymbol{Q}}_{t-1}}{\boldsymbol{x}_0\overline{\boldsymbol{Q}}_t\boldsymbol{x}_t^\top}\right)$$

Diffusion for Text

Uniform Diffusion

$$[\boldsymbol{Q}_t]_{ij} = \begin{cases} 1 - \frac{K-1}{K}\beta_t & \text{if} \quad i = j \\ \frac{1}{K}\beta_t & \text{if} \quad i \neq j \end{cases},$$

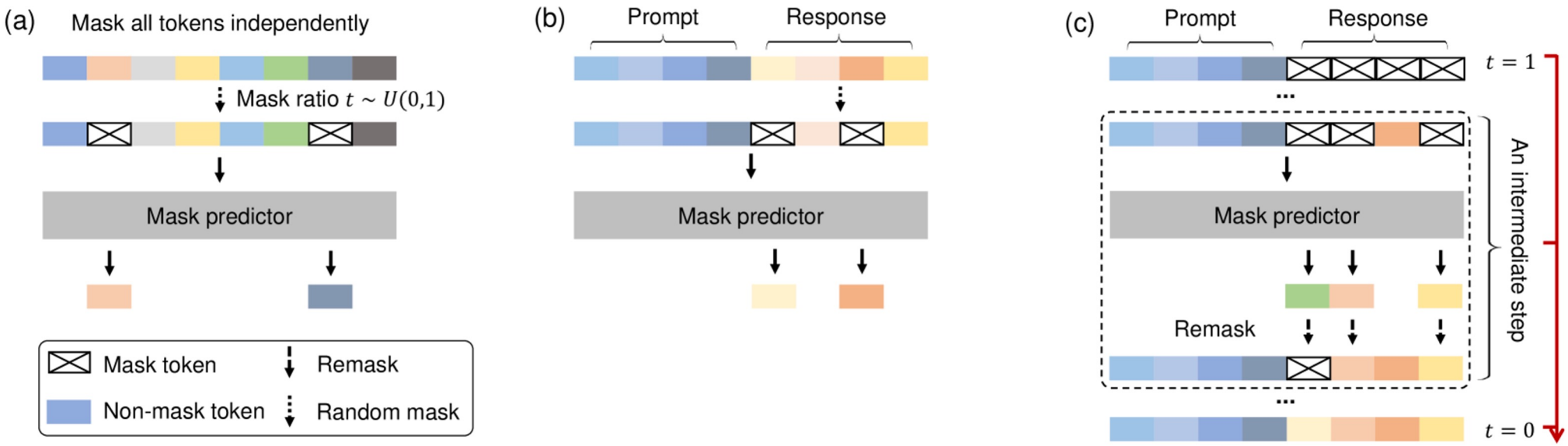
Diffusion with absorbing state

$$[\boldsymbol{Q}_t]_{ij} = \begin{cases} 1 & \text{if} \quad i = j = m \\ 1 - \beta_t & \text{if} \quad i = j \neq m \\ \beta_t & \text{if} \quad j = m, i \neq m \end{cases}$$

Diffusion for Text

$$L_\lambda = L_{\text{vb}} + \lambda \, \mathbb{E}_{q(\boldsymbol{x}_0)} \mathbb{E}_{q(\boldsymbol{x}_t|\boldsymbol{x}_0)} [-\log \widetilde{p}_\theta(\boldsymbol{x}_0|\boldsymbol{x}_t)]$$

LLaDA

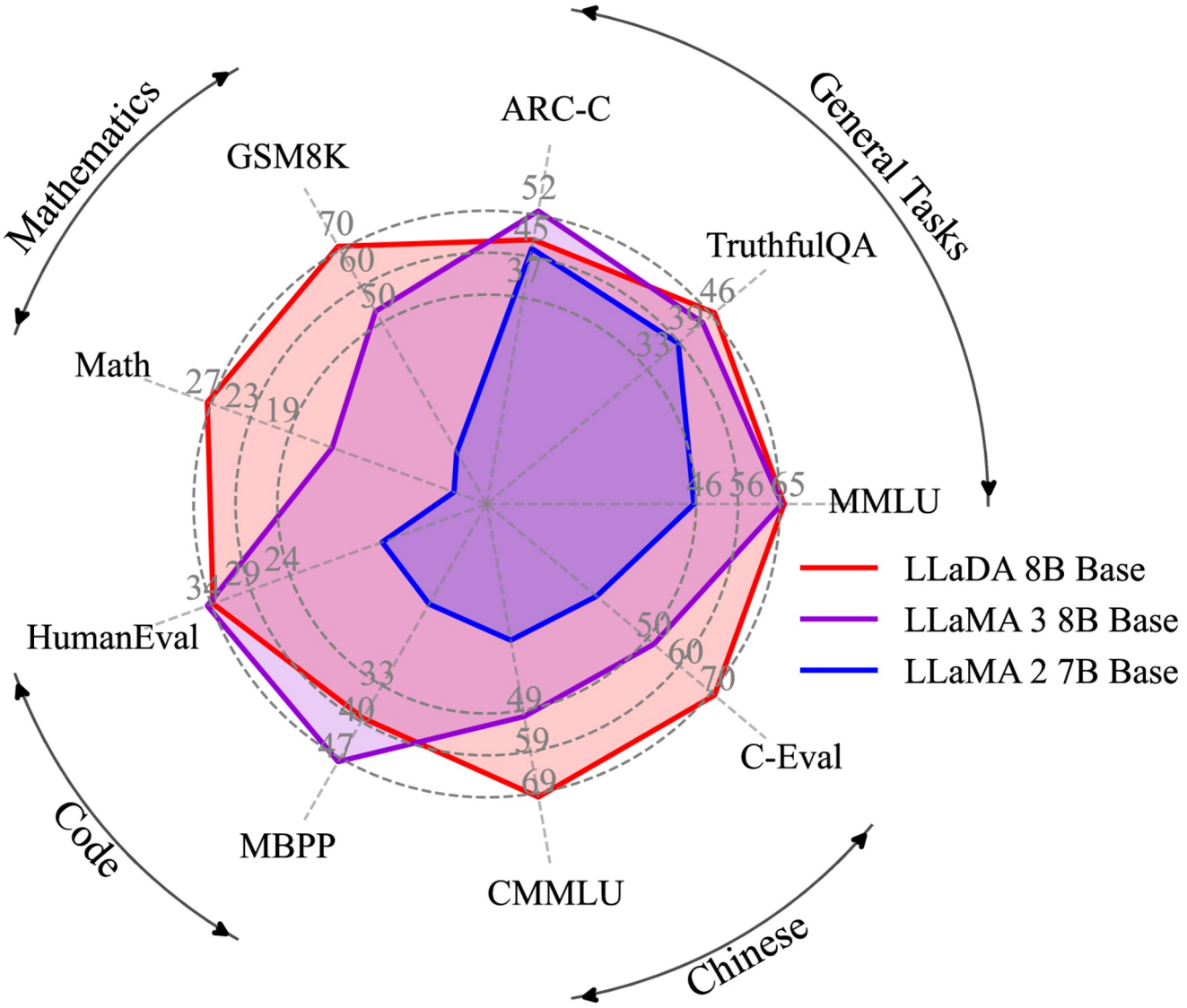

(a)
Mask all tokens independently
Mask ratio $t \sim U(0,1)$
Mask predictor
Mask token
Remask
Non-mask token
Random mask
(b)
Prompt
Response
Mask predictor
(c)
Prompt
Response
$t = 1$
Mask predictor
Remask
An intermediate step
$t = 0$

LLaDA

Predicts all masked tokens simultaneously.

$$\mathcal{L}(\theta) \triangleq -\mathbb{E}_{t,x_0,x_t}\left[\frac{1}{t}\sum_{i=1}^{L}\mathbf{1}[x_t^i=\mathrm{M}]\log p_\theta(x_0^i|x_t)\right]$$

LLaDA


Mathematics
General Tasks
Code
Chinese
GSM8K
ARC-C
TruthfulQA
Math
MMLU
HumanEval
C-Eval
MBPP
CMMLU
LLaDA 8B Base
LLaMA 3 8B Base
LLaMA 2 7B Base

Mercury

Mercury: Ultra-Fast Language Models Based on Diffusion

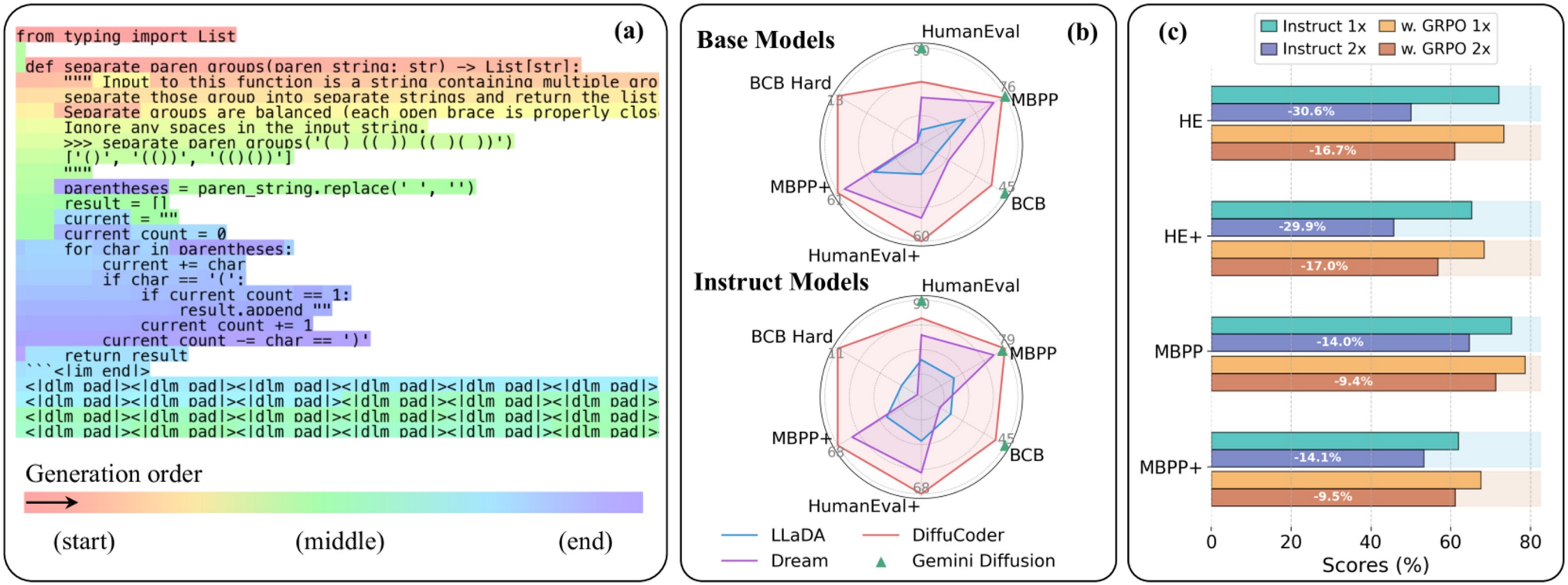
Inception Labs
Samar Khanna*, Siddhant Kharbanda*, Shufan Li*, Harshit Varma*, Eric Wang*
Sawyer Birnbaum^, Ziyang Luo^, Yanis Miraoui^, Akash Palrecha^
Stefano Ermon♯, Aditya Grover♯, Volodymyr Kuleshov♯
*^♯ equal core, cross-function, senior contributors listed alphabetically.
hello@inceptionlabs.ai

Abstract

We present Mercury, a new generation of commercial-scale large language models (LLMs) based on diffusion. These models are parameterized via the Transformer architecture and trained to predict multiple tokens in parallel. In this report, we detail Mercury Coder, our first set of diffusion LLMs designed for coding applications. Currently, Mercury Coder comes in two sizes: Mini and Small. These models set a new state-of-the-art on the speed-quality frontier. Based on independent evaluations conducted by Artificial Analysis, Mercury Coder Mini and Mercury Coder Small achieve state-of-the-art throughputs of 1109 tokens/sec and 737 tokens/sec, respectively, on NVIDIA H100 GPUs and outperform speed-optimized frontier models by up to 10× on average while maintaining comparable quality. We discuss additional results on a variety of code benchmarks spanning multiple languages and use-cases as well as real-world validation by developers on Copilot Arena, where the model currently ranks second on quality and is the fastest model overall. We also release a public API at `platform.inceptionlabs.ai` and free playground at `chat.inceptionlabs.ai`.

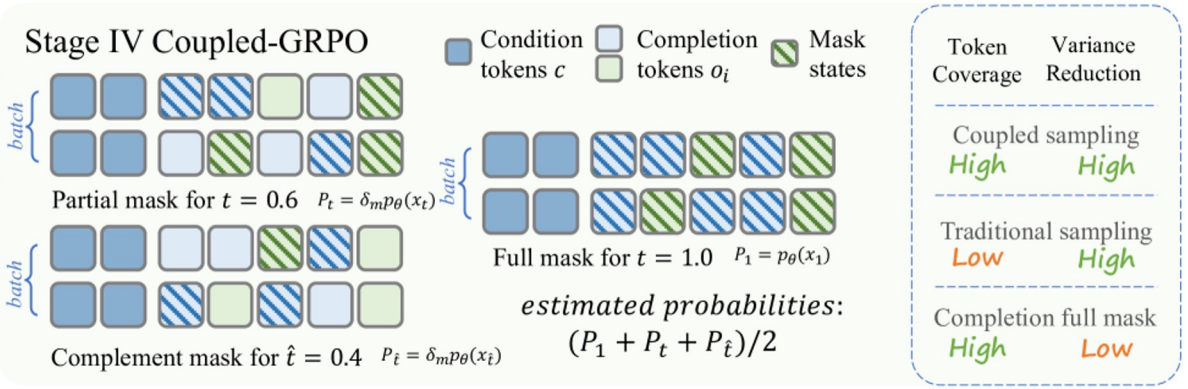
Model	HumanEval*	MBPP	EvalPlus	MultiPL-E	LCB*	BCB	Speed
Open-Weights Models							
Llama 3.1 8B Instruct	66.5	59.2	60.2	50.1	12.0	32.3	153
DeepSeek Coder V2 Lite	79.0	59.8	68.3	57.0	16.0	44.4	93
Mistral Small 3	84.8	69.6	72.3	70.1	25.0	42.7	126
Qwen 2.5 Coder 7B Instruct	88.0	80.0	79.3	75.3	9.0	41.4	195
Frontier Speed-Optimized Models							
Nova Micro	79.3	65.4	72.1	56.7	14.0	-	148
Codestral 2501	85.0	72.2	75.6	73.4	24.0	46.1	171
GPT 4o Mini	88.0	74.6	78.5	72.0	23.0	46.8	59
Claude 3.5 Haiku	86.0	78.0	75.1	72.3	31.0	45.4	61
Gemini 2.0 Flash Lite	90.0	75.0	77.3	79.5	18.0	44.4	201
Frontier Models							
DeepSeek V3	92.1	81.0	82.1	79.1	36.0	50.0	27
Claude 3.5 Sonnet	90.2	81.2	77.3	81.9	38.0	44.8	76
GPT 4o	90.2	82.2	82.4	77.6	31.0	49.9	61
Our Models							
Mercury Coder Mini	88.0	77.1	78.6	74.1	17.0	42.0	1109
Mercury Coder Small	90.0	76.6	80.4	76.2	25.0	45.5	737

DiffuCoder


(a)
from typing import List
def separate_paren_groups(paren_string: str) -> List[str]:
"""Input to this function is a string containing multiple gro
separate those group into separate strings and return the list
Separate groups are balanced (each open brace is properly clos
Ignore any spaces in the input string.
>>> separate_paren_groups('( ) (( )) (( )( ))')
['()', '(())', '(()())']
"""
parentheses = paren_string.replace(' ', '')
result = []
current = ""
current count = 0
for char in parentheses:
current += char
if char == '(':
if current count == 1:
result.append ""
current count += 1
current count -= char == ')'
return result
```<|im end|>
<|dlm pad|><|dlm pad|><|dlm pad|><|dlm pad|><|dlm pad|><|dlm pad|>
<|dlm pad|><|dlm pad|><|dlm pad|><|dlm pad|><|dlm pad|><|dlm pad|>
<|dlm pad|><|dlm pad|><|dlm pad|><|dlm pad|><|dlm pad|><|dlm pad|>
<|dlm pad|><|dlm pad|><|dlm pad|><|dlm pad|><|dlm pad|><|dlm pad|>
Generation order
(start)
(middle)
(end)
(b)
Base Models
HumanEval
MBPP
BCB
HumanEval+
MBPP+
BCB Hard
90
76
45
60
61
13
Instruct Models
HumanEval
MBPP
BCB
HumanEval+
MBPP+
BCB Hard
90
79
45
68
68
11
LLaDA
Dream
DiffuCoder
Gemini Diffusion
(c)
Instruct 1x
Instruct 2x
w. GRPO 1x
w. GRPO 2x
HE
HE+
MBPP
MBPP+
-30.6%
-16.7%
-29.9%
-17.0%
-14.0%
-9.4%
-14.1%
-9.5%
0
20
40
60
80
Scores (%)
```

DiffuCoder

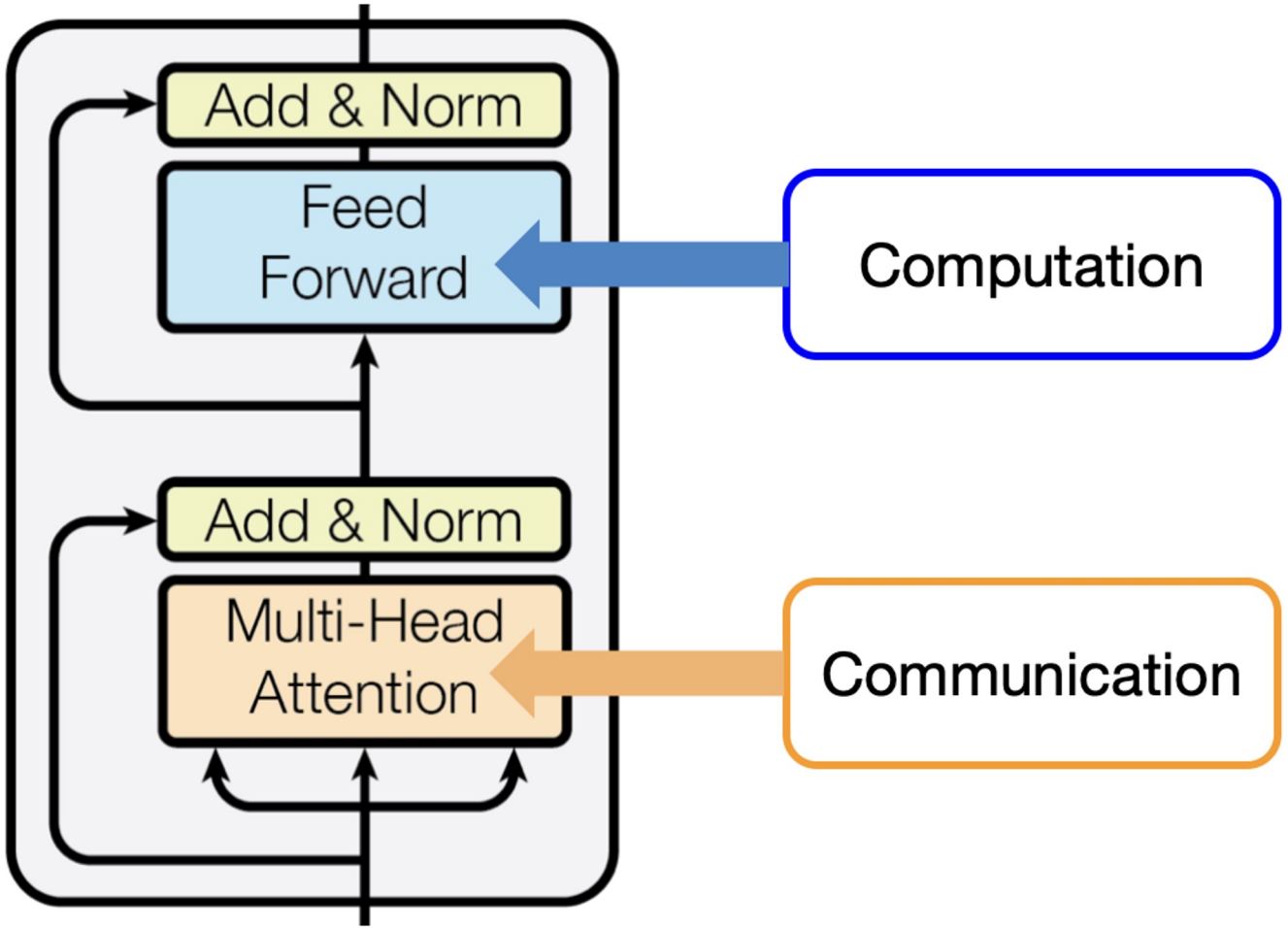
Stages	Stage I	Stage II	Stage III
Description	Adaptation-pretraining	Mid-training	Instruction tuning
Training Data	RefineCode mixture	OpenCoder annealing	OpenCoder SFT
Training Strategy	Packed Unconditional	Packed Unconditional	Padded Conditional

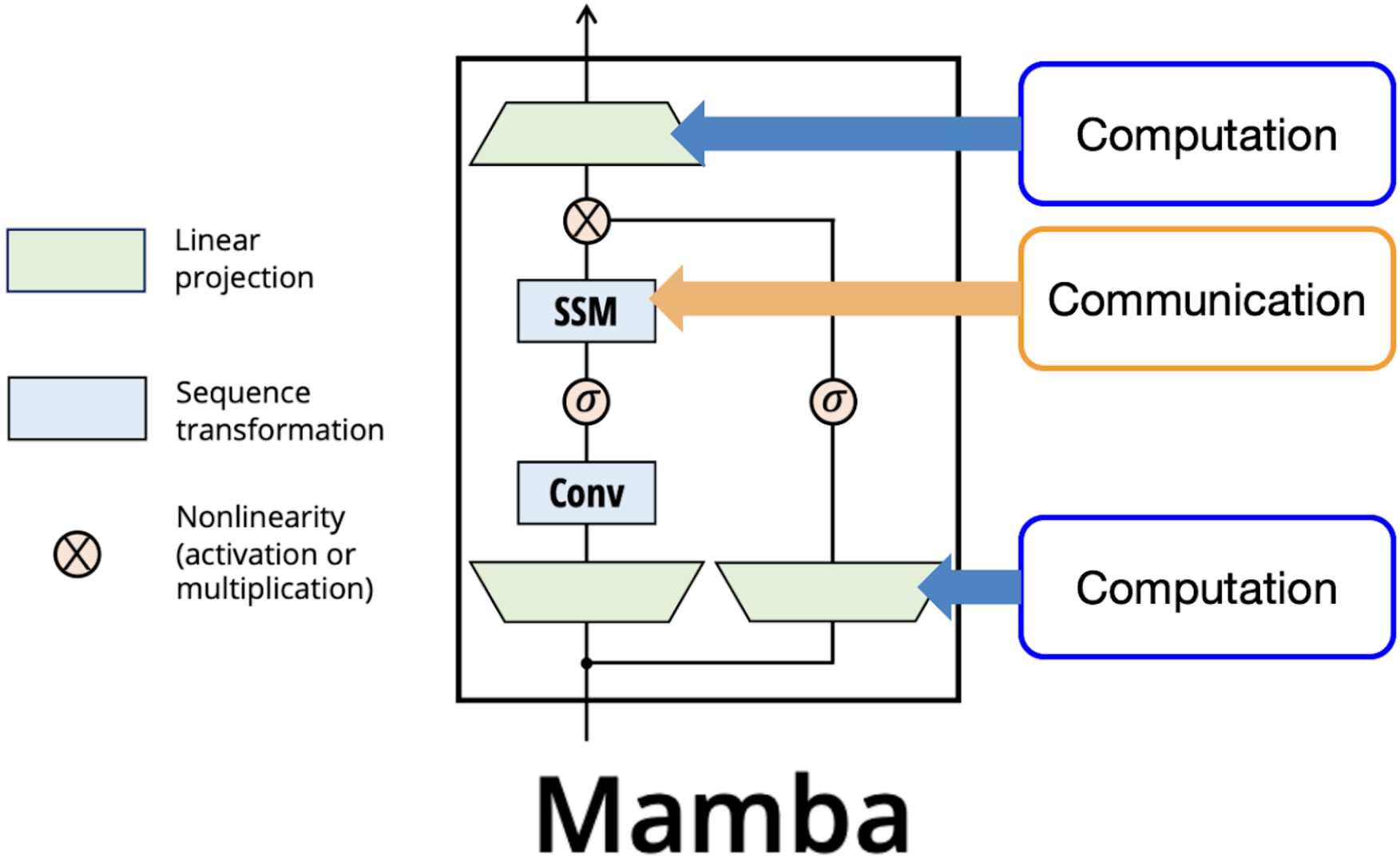


Model	HumanEval		MBPP		EvalPlus	BigCodeBench (C)		Avg.
	-	Plus	-	Plus		Full	Hard	
Base Models								
Qwen2.5-Coder	61.6	51.8	75.9	61.4	56.6	46.1	16.2	52.2
OpenCoder*	66.5	63.4	79.9	70.4	66.9	40.5	9.5	55.0
LLaDA	35.4	30.5	50.1	42.1	36.3	18.9	4.1	30.2
Dream	56.7	50.0	68.7	57.4	53.7	23.6	4.1	43.4
DiffuCoder	**67.1**	**60.4**	74.2	60.9	**60.6**	40.2	**12.8**	**52.6**
Instruct Models								
Qwen2.5-Coder-Instruct	90.2	85.4	83.9	72.0	78.7	50.7	21.6	67.3
Qwen2.5-Coder+SFT	82.9+21.3	75.6+23.8	80.1+4.2	66.1+4.7	70.9+14.3	46.9+0.8	16.2–0.0	61.3+9.1
OpenCoder-Instruct*	83.5+17.0	78.7+15.3	79.1–0.8	69.0–1.4	73.9+7.0	40.3–0.2	16.9+7.4	61.3+6.3
LLaDA-Instruct	35.4–0.0	31.7+1.2	31.5–18.6	28.6–13.5	30.2–6.1	16.5–2.4	2.7–1.4	24.4–5.8
Dream-Instruct	57.9+1.2	53.7+3.7	68.3–0.4	56.1–1.3	54.9+1.2	10.6–13.0	0.7–3.4	41.2–2.2
DiffuCoder-Instruct	72.0+4.9	65.2+4.8	75.1+0.9	61.9+1.0	63.6+3.0	35.7–4.5	12.2–0.6	53.7+1.1
+ coupled-GRPO	73.2+6.1	68.3+7.9	**78.6**+4.4	67.5+6.6	67.9+7.3	40.4+0.2	10.8–2.0	56.5+3.9
Commercial Models								
GPT 4o*	90.2	–	82.2	–	82.4	49.9	–	–
Mercury*	90.0	–	77.1	–	80.4	45.5	–	–
Gemini Diffusion*	89.6	–	76.0	–	–	45.4	–	–

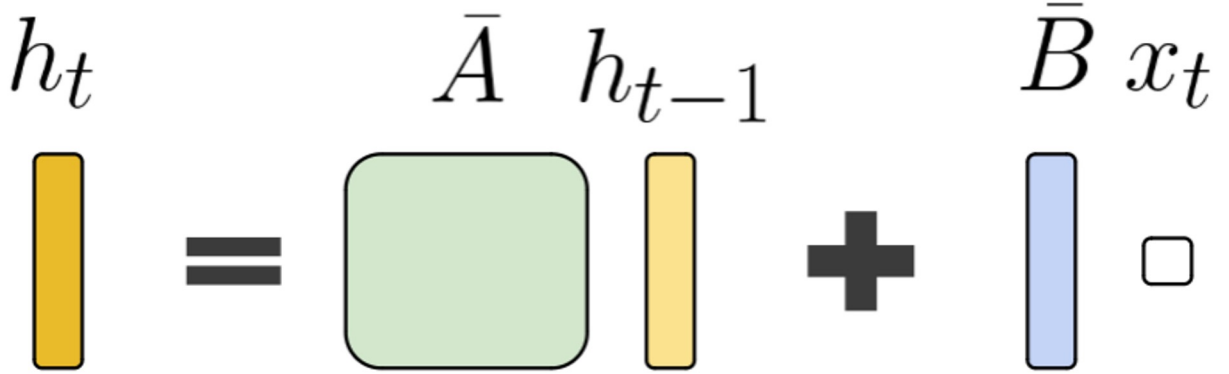
State Space Models

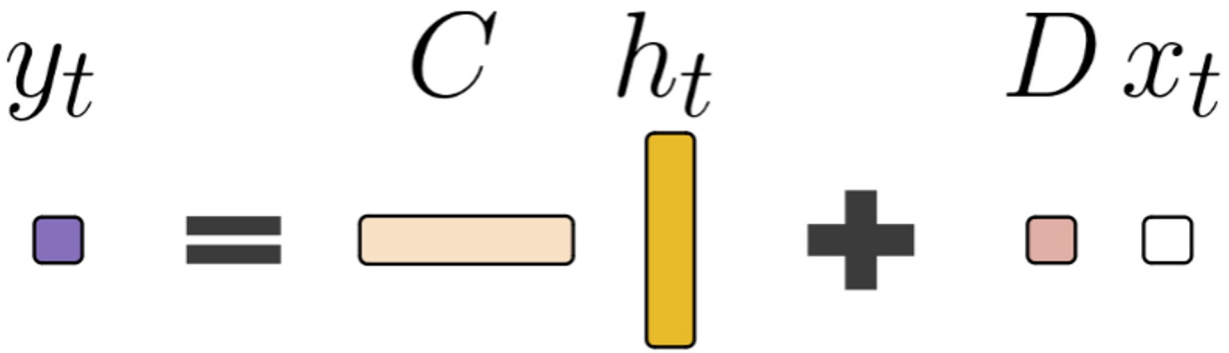
Transformer

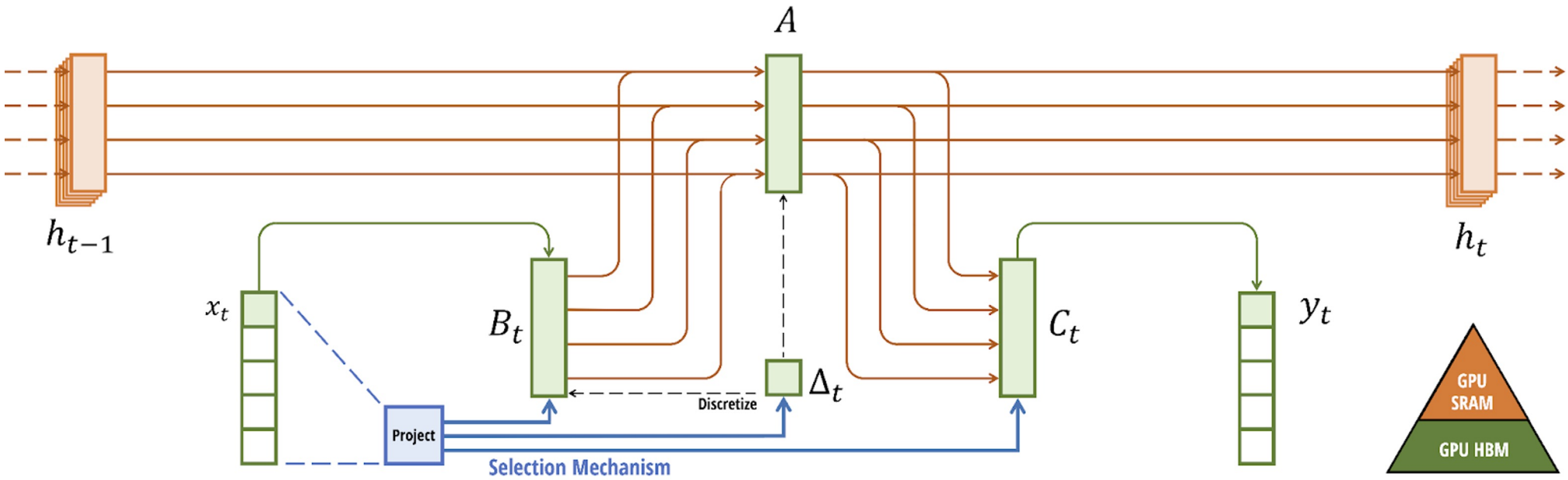



Linear projection
Sequence transformation
Nonlinearity (activation or multiplication)
SSM
Conv
Computation
Communication
Computation
Mamba

Equations






Selective State Space Model
with Hardware-aware State Expansion
A
h_{t-1}
h_t
x_t
B_t
C_t
y_t
Δ_t
Discretize
Project
Selection Mechanism
GPU SRAM
GPU HBM

State Selection

Algorithm 1 SSM (S4)

Input: $x : (\mathtt{B}, \mathtt{L}, \mathtt{D})$
Output: $y : (\mathtt{B}, \mathtt{L}, \mathtt{D})$

$\boldsymbol{A} : (\mathtt{D}, \mathtt{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
$\boldsymbol{B} : (\mathtt{D}, \mathtt{N}) \leftarrow$ Parameter
$\boldsymbol{C} : (\mathtt{D}, \mathtt{N}) \leftarrow$ Parameter
$\Delta : (\mathtt{D}) \leftarrow \tau_\Delta(\text{Parameter})$
$\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathtt{D}, \mathtt{N}) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
$y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-invariant: recurrence or convolution
return y

Algorithm 2 SSM + Selection (S6)

Input: $x : (\mathtt{B}, \mathtt{L}, \mathtt{D})$
Output: $y : (\mathtt{B}, \mathtt{L}, \mathtt{D})$

$\boldsymbol{A} : (\mathtt{D}, \mathtt{N}) \leftarrow$ Parameter
▷ Represents structured $N \times N$ matrix
$\boldsymbol{B} : (\mathtt{B}, \mathtt{L}, \mathtt{N}) \leftarrow s_B(x)$
$\boldsymbol{C} : (\mathtt{B}, \mathtt{L}, \mathtt{N}) \leftarrow s_C(x)$
$\Delta : (\mathtt{B}, \mathtt{L}, \mathtt{D}) \leftarrow \tau_\Delta(\text{Parameter} + s_\Delta(x))$
$\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}} : (\mathtt{B}, \mathtt{L}, \mathtt{D}, \mathtt{N}) \leftarrow \text{discretize}(\Delta, \boldsymbol{A}, \boldsymbol{B})$
$y \leftarrow \text{SSM}(\overline{\boldsymbol{A}}, \overline{\boldsymbol{B}}, \boldsymbol{C})(x)$
▷ Time-varying: recurrence (*scan*) only
return y

Mamba 3

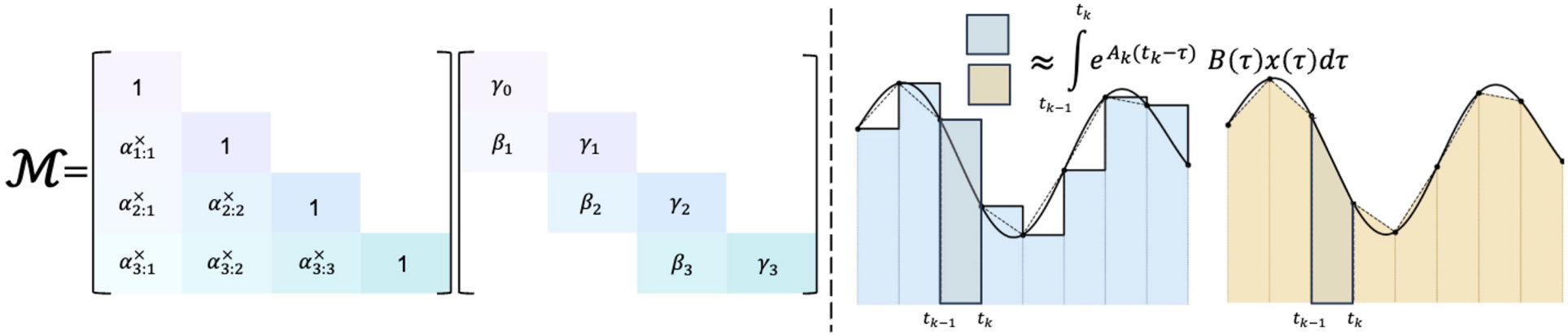
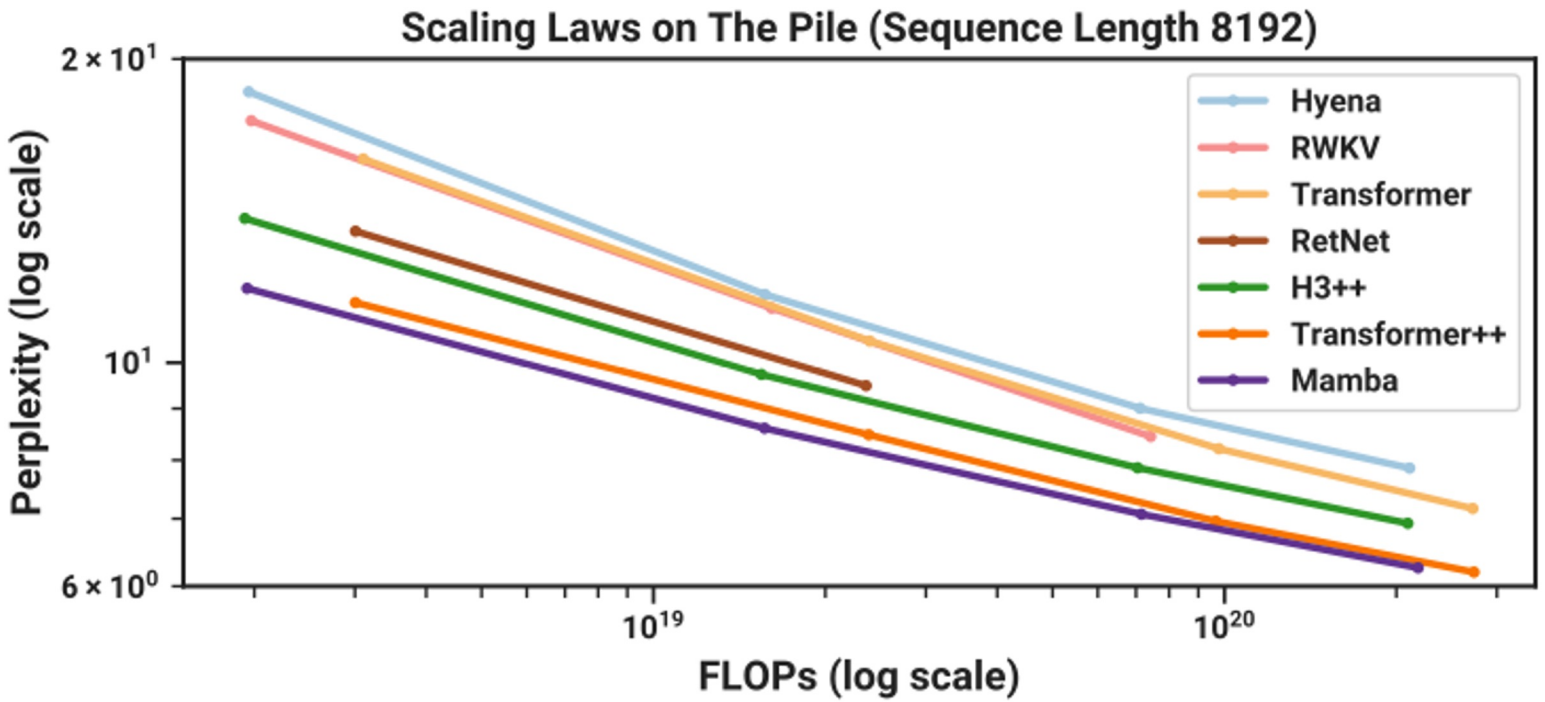


Figure 1: **Left:** The structured mask induced by the generalized trapezoid rule is a product of the decay and convolutional mask. **Right:** Euler (hold endpoint) vs trapezoidal rule (average endpoints).

Table 1: Downstream language modeling evaluations on models trained with 100B FineWeb-Edu tokens. Best results for each size are **bolded**, and second best are underlined. All models are trained with the same procedure. Mamba-3 outperforms Mamba-2 and others at every model scale.

Model	FW-Edu ppl ↓	LAMB. ppl ↓	LAMB. acc ↑	HellaS. acc_n ↑	PIQA acc ↑	Arc-E acc ↑	Arc-C acc_n ↑	WinoGr. acc ↑	OBQA acc_n ↑	Average acc ↑
Transformer-180M	16.89	45.0	32.5	39.0	**67.1**	59.8	27.9	51.2	21.8	42.8
Gated DeltaNet-180M	16.61	**35.9**	**33.7**	40.2	66.8	59.6	**28.5**	51.2	21.6	43.1
Mamba-2-180M	16.76	41.8	30.9	40.1	66.8	60.1	27.3	**52.0**	**23.2**	42.9
Mamba-3-180M	**16.59**	37.7	32.5	**40.8**	66.1	**61.5**	27.9	**52.0**	22.8	**43.4**
Transformer-440M	13.03	21.2	**41.7**	50.5	69.9	67.6	34.6	**56.7**	**26.0**	49.6
Gated DeltaNet-440M	13.12	**19.0**	40.4	50.5	70.5	67.5	34.0	55.3	25.8	49.1
Mamba-2-440M	13.00	19.6	40.8	**51.7**	70.6	68.8	**35.0**	54.1	**26.0**	49.6
Mamba-3-440M	**12.87**	19.6	40.2	**51.7**	**71.9**	**68.9**	34.4	55.8	**26.0**	**49.8**
Transformer-820M	11.42	15.0	44.7	57.2	72.6	71.6	39.2	57.7	26.8	52.8
Gated DeltaNet-820M	11.39	**12.7**	47.1	57.5	72.6	72.5	38.8	57.9	**30.6**	53.9
Mamba-2-820M	11.35	13.8	45.0	58.1	72.5	72.3	38.7	56.8	30.2	53.4
Mamba-3-820M	**11.23**	12.9	**47.2**	**58.8**	**73.6**	**72.7**	**40.2**	**58.4**	30.0	**54.4**
Transformer-1.5B	10.51	11.1	**50.3**	60.6	73.8	74.0	40.4	58.7	29.6	55.4
Gated DeltaNet-1.5B	10.51	**10.8**	49.9	60.5	**74.3**	73.3	40.4	**61.5**	30.4	55.7
Mamba-2-1.5B	10.47	12.0	47.8	61.4	73.6	75.3	41.8	57.5	**32.6**	55.7
Mamba-3-1.5B	**10.35**	10.9	49.4	**61.9**	73.6	**75.9**	**42.7**	59.4	32.0	**56.4**

Results


Scaling Laws on The Pile (Sequence Length 8192)
Perplexity (log scale)
2 × 10^1
10^1
6 × 10^0
10^19
10^20
FLOPs (log scale)
Hyena
RWKV
Transformer
RetNet
H3++
Transformer++
Mamba

New RNNs

Complexity

Model	Time	Space
Transformer	$O(T^2 d)$	$O(T^2 + Td)$
Reformer	$O(T \log T d)$	$O(T \log T + Td)$
Performer	$O(Td^2 \log d)$	$O(Td \log d + d^2 \log d)$
Linear Transformers	$O(Td^2)$	$O(Td + d^2)$
AFT-full	$O(T^2 d)$	$O(Td)$
AFT-local	$O(Tsd)$	$O(Td)$
MEGA	$O(cTd)$	$O(cd)$
RWKV (ours)	$O(\mathbf{Td})$	$O(\mathbf{d})$

Table 1: Inference complexity comparison with different Transformers. Here T denotes the sequence length, d the feature dimension, c is MEGA's chunk size of quadratic attention, and s is the size of a local window for AFT.

LSTM

$$f_t = \sigma_g(W_f x_t + U_f h_{t-1} + b_f), \quad (1)$$

$$i_t = \sigma_g(W_i x_t + U_i h_{t-1} + b_i), \quad (2)$$

$$o_t = \sigma_g(W_o x_t + U_o h_{t-1} + b_o), \quad (3)$$

$$\tilde{c}_t = \sigma_c(W_c x_t + U_c h_{t-1} + b_c), \quad (4)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad (5)$$

$$h_t = o_t \odot \sigma_h(c_t). \quad (6)$$

Attention

$$\text{Attn}(Q, K, V)_t = \frac{\sum_{i=1}^{T} e^{q_t^\top k_i} \odot v_i}{\sum_{i=1}^{T} e^{q_t^\top k_i}}. \quad (8)$$

AFT (Zhai et al., 2021), alternately formulates

$$\text{Attn}^+(W, K, V)_t = \frac{\sum_{i=1}^{t} e^{w_{t,i}+k_i} \odot v_i}{\sum_{i=1}^{t} e^{w_{t,i}+k_i}}, \quad (9)$$

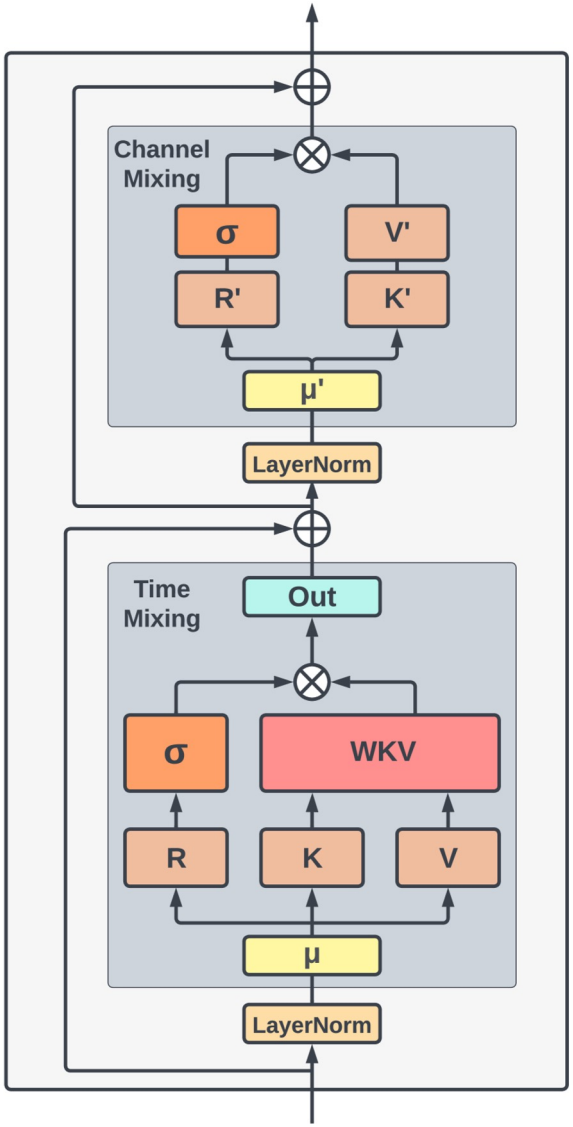
where $\{w_{t,i}\} \in R^{T\times T}$ is the learned pair-wise position biases, and each $w_{t,i}$ is a scalar.

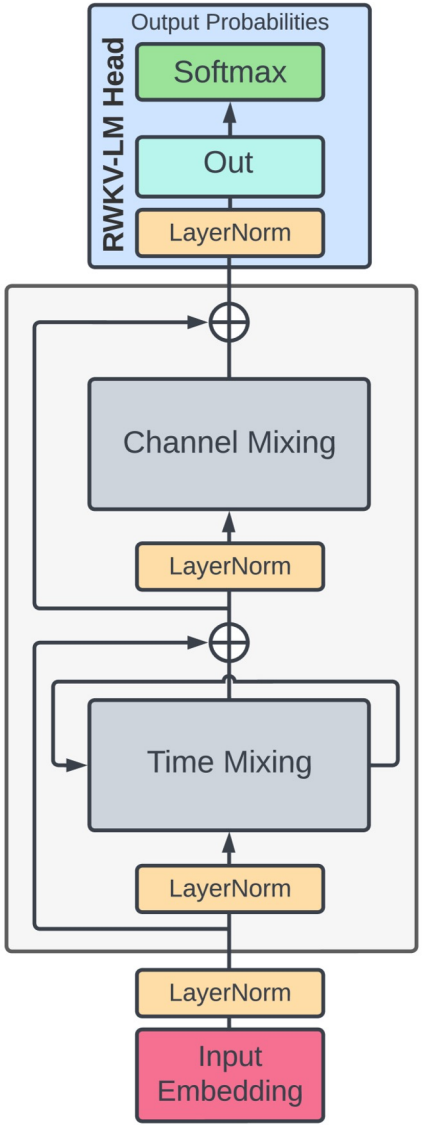
$$w_{t,i} = -(t-i)w, \quad (10)$$

RWKV

- R: The Receptance vector acts as the receiver of past information.
- W : The Weight signifies the positional weight decay vector, a trainable parameter within the model.
- K: The Key vector performs a role analogous to K in traditional attention mechanisms.
- V : The Value vector functions similarly to Vin conventional attention processes.

RWKV


Channel Mixing
σ
V'
R'
K'
μ'
LayerNorm
Time Mixing
Out
σ
WKV
R
K
V
μ
LayerNorm


Output Probabilities
RWKV-LM Head
Softmax
Out
LayerNorm
Channel Mixing
LayerNorm
Time Mixing
LayerNorm
LayerNorm
Input Embedding

Token Shift

$$r_t = W_r \cdot (\mu_r \odot x_t + (1 - \mu_r) \odot x_{t-1}), \quad (11)$$
$$k_t = W_k \cdot (\mu_k \odot x_t + (1 - \mu_k) \odot x_{t-1}), \quad (12)$$
$$v_t = W_v \cdot (\mu_v \odot x_t + (1 - \mu_v) \odot x_{t-1}), \quad (13)$$

$$r'_t = W'_r \cdot (\mu'_r \odot x_t + (1 - \mu'_r) \odot x_{t-1}), \quad (14)$$
$$k'_t = W'_k \cdot (\mu'_k \odot x_t + (1 - \mu'_k) \odot x_{t-1}). \quad (15)$$

WKV Operator

$$wkv_t = \frac{\sum_{i=1}^{t-1} e^{-(t-1-i)w+k_i} \odot v_i + e^{u+k_t} \odot v_t}{\sum_{i=1}^{t-1} e^{-(t-1-i)w+k_i} + e^{u+k_t}}. \tag{16}$$

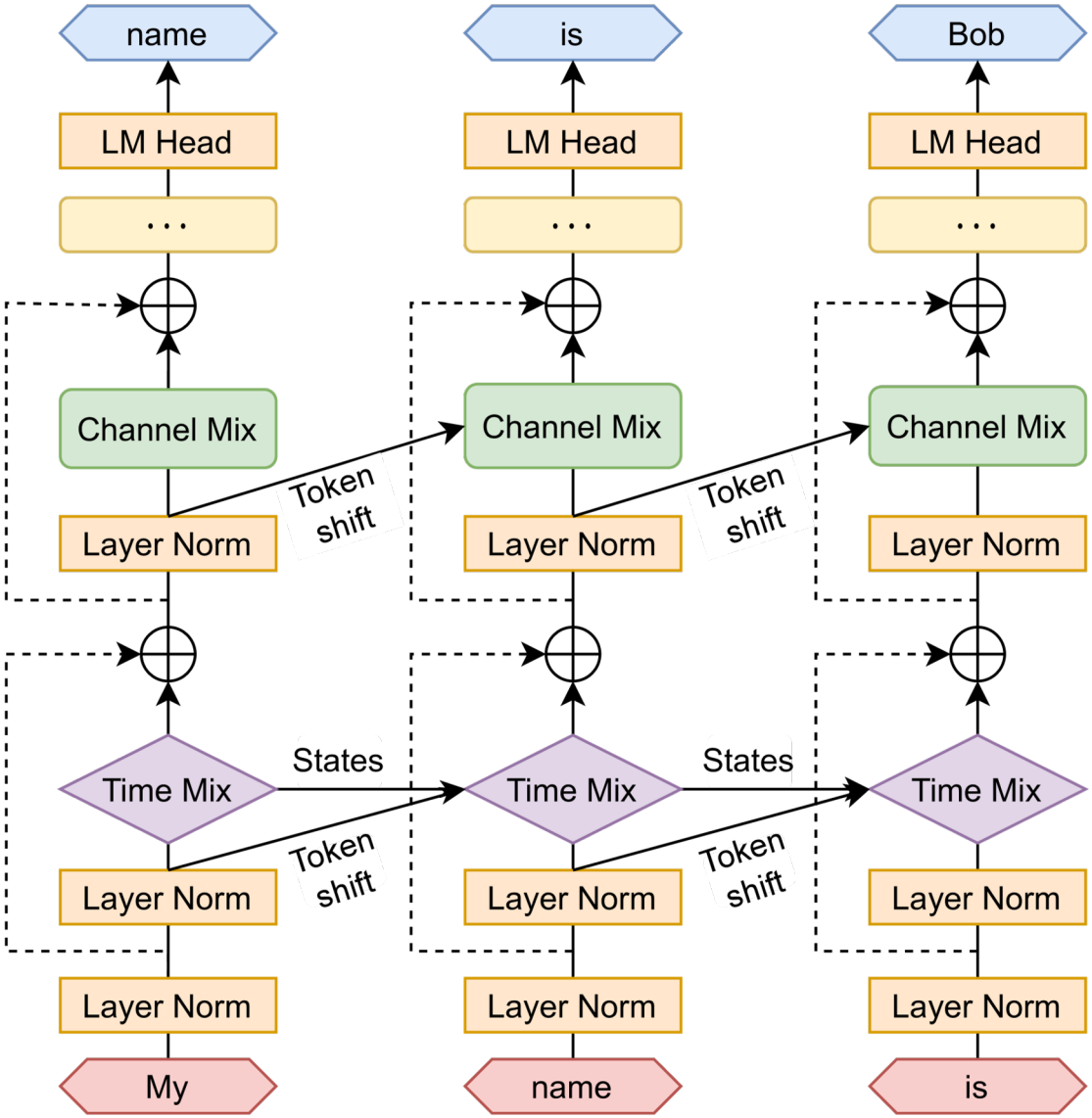
Output Gate

$$o_t = W_o \cdot (\sigma(r_t) \odot wkv_t). \tag{17}$$

In the channel-mixing block, a similar operation is performed:

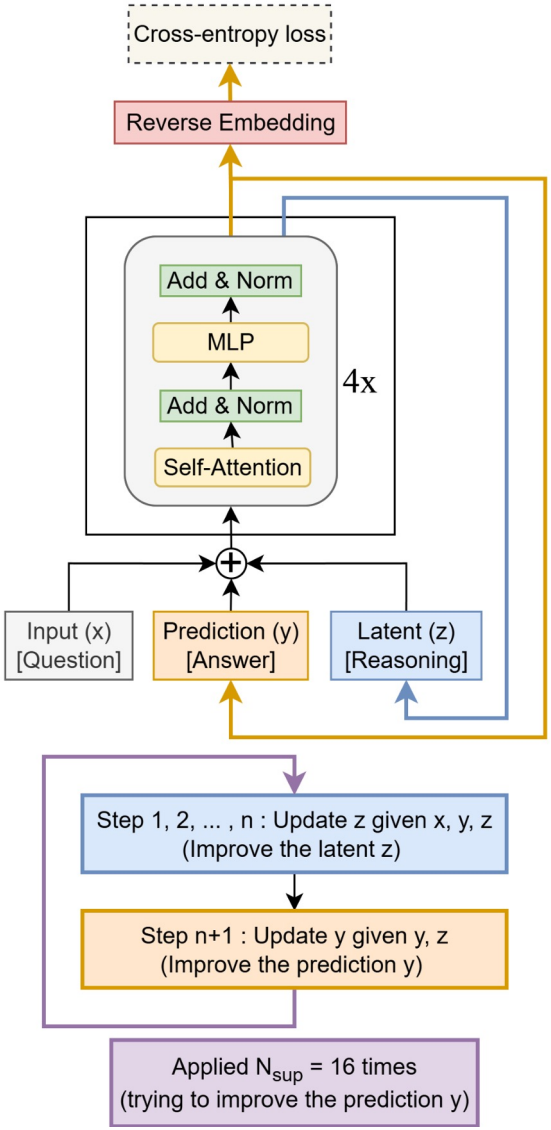
$$o'_t = \sigma(r'_t) \odot (W'_v \cdot \max(k'_t, 0)^2), \tag{18}$$

Token Shift


name
is
Bob
LM Head
LM Head
LM Head
...
...
...
Channel Mix
Channel Mix
Channel Mix
Token shift
Token shift
Layer Norm
Layer Norm
Layer Norm
Time Mix
Time Mix
Time Mix
States
States
Token shift
Token shift
Layer Norm
Layer Norm
Layer Norm
Layer Norm
Layer Norm
Layer Norm
My
name
is

Exotic LMs

Hierarchical Reasoning Model



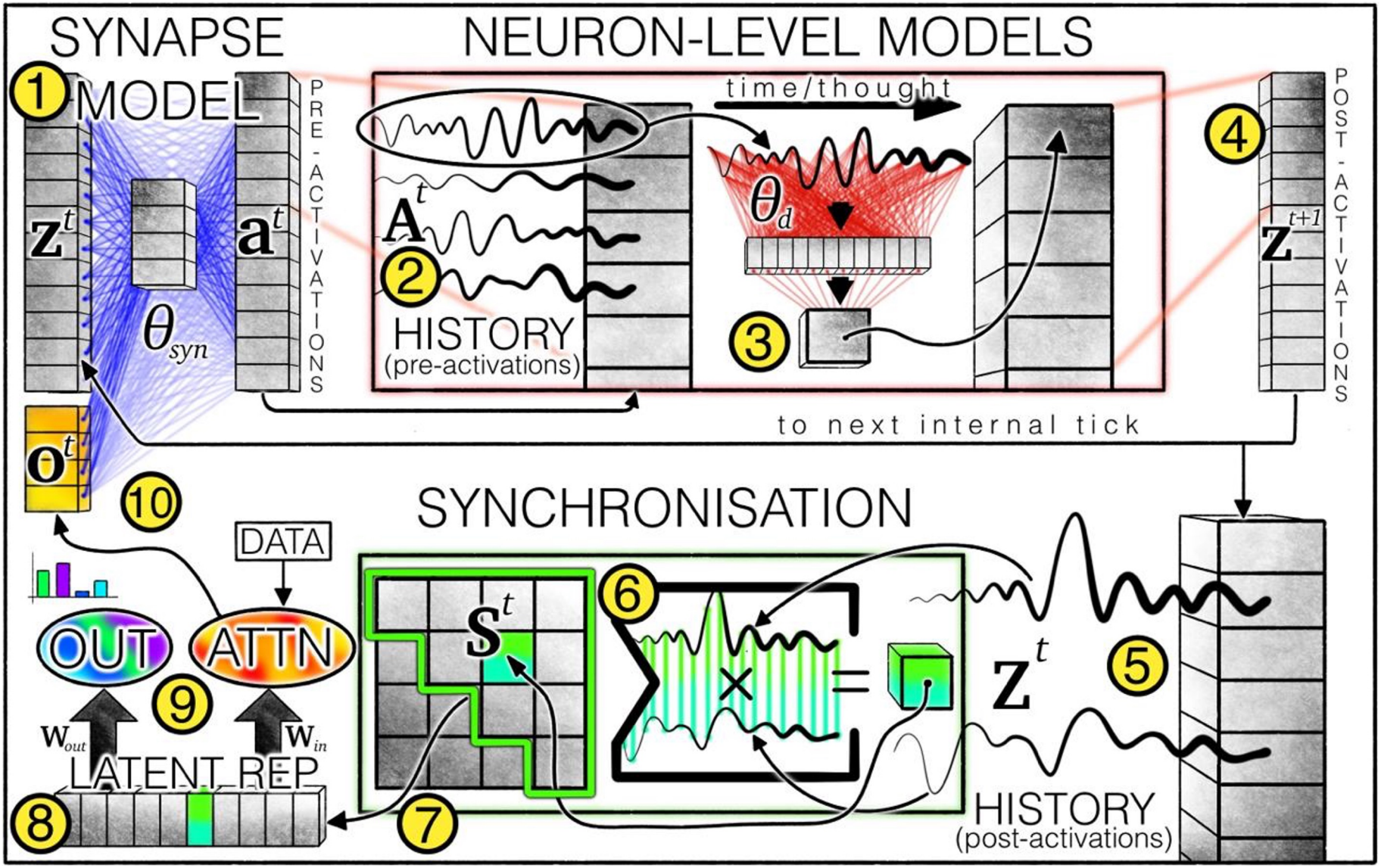

```python
def hrm(z, x, n=2, T=2): # hierarchical reasoning
    zH, zL = z
    with torch.no_grad():
        for i in range(nT - 2):
            zL = L_net(zL, zH, x)
            if (i + 1) % T == 0:
                zH = H_net(zH, zL)
    # 1-step grad
    zL = L_net(zL, zH, x)
    zH = H_net(zH, zL)
    return (zH, zL), output_head(zH), Q_head(zH)

def ACT_halt(q, y_hat, y_true):
    target_halt = (y_hat == y_true)
    loss = 0.5*binary_cross_entropy(q[0], target_halt)
    return loss

def ACT_continue(q, last_step):
    if last_step:
        target_continue = sigmoid(q[0])
    else:
        target_continue = sigmoid(max(q[0], q[1]))
    loss = 0.5*binary_cross_entropy(q[1], target_continue)
    return loss

# Deep Supervision
for x_input, y_true in train_dataloader:
    z = z_init
    for step in range(N_sup): # deep supervision
        x = input_embedding(x_input)
        z, y_pred, q = hrm(z, x)
        loss = softmax_cross_entropy(y_pred, y_true)
        # Adaptive computational time (ACT) using Q-learning
        loss += ACT_halt(q, y_pred, y_true)
        _, _, q_next = hrm(z, x) # extra forward pass
        loss += ACT_continue(q_next, step == N_sup - 1)
        z = z.detach()
        loss.backward()
        opt.step()
        opt.zero_grad()
        if q[0] > q[1]: # early-stopping
            break
```

Figure 2. Pseudocode of Hierarchical Reasoning Models (HRMs).


SYNAPSE
NEURON-LEVEL MODELS
1
MODEL
PRE-ACTIVATIONS
z^t
a^t
θ_{syn}
time/thought
A^t
2
HISTORY
(pre-activations)
θ_d
3
4
z^{t+1}
POST-ACTIVATIONS
to next internal tick
o^t
10
SYNCHRONISATION
DATA
OUT
ATTN
9
W_{out}
W_{in}
LATENT REP
8
S^t
6
7
z^t
5
HISTORY
(post-activations)

Conclusion

- Transformers are limited
- We do not know how to avoid this limitation
- Old ideas and new ideas could work

Thank you for your attention!

Valentin Malykh
email: valentin.malykh@phystech.edu
telegram: @madrugado

